If already registered, click to check your payment status

Course outline

About NPTEL ()

How does an NPTEL online course work? ()

Week 1 ()

Week 2 ()

Week 3 ()

week 4 ()

Week 5 ()

Week 6 ()

Week 7 ()

Snakes and Ladders - Not on the Board (unit?unit=143&lesson=144)

Snakes and Ladders - Not on the Board - Part 01 (unit?

Thank you for taking the Week 7 : Assignment 7.

Week 7 : Assignment 7

Your last recorded submission was on 2025-09-10, 22:32 IST **Due date: 2025-09-10, 23:59 IST.**

Refer to the code discussed in the "Joy of Computing using Python" lecture on implementing the Snake and Ladder game.

Ankan and Dilrez are playing a classic game of Snake and Ladder –but this time, it's implemented in Python! The game logic is based on the version you've seen in the lecture, where snakes pull you down and ladders push you up. The game board ranges from position 1 to 100.

The snake and ladder positions are stored in **dictionaries**. For instance:
snakes = {16: 6, 98: 78}
ladders = {1: 38, 80: 100}

Each player begins at position 0. In each turn, a player rolls a die (using random.randint(1, 6)) and moves forward. If they land on the bottom of a ladder, they climb up; if they land on a snake's mouth, they slide down. The game alternates between Ankan and Dilrez using a turn variable, and ends when any player reaches exactly 100.

Two key functions, check_ladder(position) and check_snake(position), determine whether the player hits a ladder or a snake after moving.

A function reached_end(position) is used to check if a player has reached position 100. A separate function is also written to open and display an image of the board using the **PIL** library.

1) In the function play(), what is the significance of the variable turn? ***1 point***

- It tracks the number of dice rolls
- It ensures alternating turns between the two players
- It ends the game after a fixed number of turns
- It stores the winning score

2) What happens in the code when a player rolls a number that moves them past position 100? ***1 point***

- The game ends and they win

Assessment submitted.
X

unit=143&lesson=145)
Snakes and Ladders - Not on the Board - Part 02 (unit?unit=143&lesson=146)
Snakes and Ladders - Not on the Board - Part 03 (unit?unit=143&lesson=147)
Snakes and Ladders - Not on the Board - Part 04 (unit?unit=143&lesson=148)
Snakes and Ladders - Not on the Board - Part 05 (unit?unit=143&lesson=149)
Snakes and Ladders - Not on the Board - Part 06 (unit?unit=143&lesson=150)
Spiral Traversing - Let's Animate (unit?unit=143&lesson=151)
Spiral Traversing - Let's Animate - Part 01 (unit?unit=143&lesson=152)
Spiral Traversing - Let's Animate - Part 02 (unit?unit=143&lesson=153)
Spiral Traversing - Let's Animate - Part 03 (unit?

- The player's position resets to 0
- (selected) The player stays in the same position (doesn't move)
- The player is forced to roll again

3) What is the purpose of the following lines in the play() function? ***1 point***

```
if c == 0:
    print(p1_name, 'scored', pp1)
    print(p2_name, 'scored', pp2)
    print('Quitting the game. Thanks for playing.')
    break
```

- To restart the game
- (selected) To display final scores and exit if the player chooses not to continue
- To check if the player reached a snake
- To decide the winner

4) If Ankan is at position 47 and the snakes dictionary contains 47:26, what will happen immediately after his dice roll moves him to 47? ***1 point***

- He will stay at 47
- He will move to 100
- (selected) He will go to 26
- The game will end

5) What does the function show_board() attempt to do? ***1 point***

- Display current scores
- Roll the dice for both players
- (selected) Open and display the game board image
- Initialize the snake and ladder dictionaries

6) Which of these functions is called after a player moves and before their position is printed? ***1 point***

- show_board()
- play()
- (selected) check_ladder() and check_snake()
- reached_end() only

7) How many times is the line random.randint(1, 6) evaluated in one full round of both players' turns? ***1 point***

- Once
- (selected) Twice
- Four times
- Depends on the player positions

8) Which part of the code ensures that the game stops immediately when a player reaches exactly 100? ***1 point***

Assessment submitted.
X

unit=143&lesson=154)
Spiral Traversing - Let's Animate - Part 04 (unit?unit=143&lesson=155)
Spiral Traversing - Let's Animate - Part 05 (unit?unit=143&lesson=156)
Spiral Traversing - Let's Animate - Part 06 (unit?unit=143&lesson=157)
Spiral Traversing - Let's Animate - Part 07 (unit?unit=143&lesson=158)
GPS - Track the route (unit?unit=143&lesson=159)
GPS - Track the route - Part 01 (unit?unit=143&lesson=160)
GPS - Track the route - Part 02 (unit?unit=143&lesson=161)
GPS - Track the route - Part 03 (unit?unit=143&lesson=162)
GPS - Track the route - Part 04 (unit?unit=143&lesson=163)
Week 7 Feedback Form: The Joy of Computing

- if pp1 > end:
- if pp1 == 100:
- if reached_end(pp1):
- if check_ladder(pp1):

9) Consider the following ladders dictionary (in the format bottom: top): ladders = {1: 38, 4: 14, 9: 31, 21: 42, 28: 84, 36: 44, 51: 67, 71: 91, 80: 100} ***1 point***

If a player is on **27**, rolls a **1**, and lands on **28**, what will happen next?

- The player will stay on 28 because it's not a ladder base
- The player climbs the ladder to 84
- The player must roll again because 28 is a special position
- The code throws an error because 28 is not handled explicitly

10) ***1 point***

```
while True:
    if turn % 2 == 0:
        print(f"\n{p1_name}'s turn")
        ...
```

Why is the condition turn % 2 == 0 used in this part of the code?

- To check if the game has reached the second round
- To make sure player 2 only plays after player 1 wins
- To alternate turns between the two players using the value of turn
- To skip the dice roll for even-numbered turns

Dilrez, the ever-curious adventurer, had a habit of collecting strange and shiny things during her journeys.

One day, the village elder gifted her **two magical containers** — one called a **Backpack** and the other a **Pouch**.

The elder explained:

“Dilrez, these are no ordinary containers.

The **Backpack** holds items in a fixed order, and once packed, nothing can be changed.

The **Pouch**, on the other hand, only keeps unique items and doesn’t care about order.”

Dilrez was intrigued.

Backpack = Tuple

Dilrez packed it like this:

backpack = ("map", "compass", "torch")

He checked backpack[0] → 'map'.

Assessment submitted.
X

using Python (unit?unit=143&lesson=164)

Quiz: Week 7 : Assignment 7 (assessment?name=603)

Week 7 : Programming Assignment 1 (/noc25_cs103/progassignment?name=584)

Week 7 : Programming Assignment 2 (/noc25_cs103/progassignment?name=585)

Week 7 : Programming Assignment 3 (/noc25_cs103/progassignment?name=602)

Week 8 ()

Text Transcripts ()

Download Videos ()

Problem Solving Session - July 2025 ()

But when he tried:

backpack[0] = "magic map"

He got an error!

Tuples are **ordered, immutable**, and allow **duplicates**.
Syntax: () (round brackets)

Pouch = **Set**

Dilrez filled his pouch:

pouch = {"coin", "potion", "coin"}

He expected 3 items, but got:

{'coin', 'potion'}

He added and removed items freely:

pouch.add("gem")

pouch.remove("coin")

Sets are **unordered, mutable**, and store **unique** items.
Syntax: {} (curly braces)

11) What will be the output of the following code? ***1 point***

```
backpack = ("map", "compass", "torch")
print(backpack[1])
```

- "map"
- "torch"
- (selected) "compass"
- IndexError

12) What will happen when Dilrez runs this code? ***1 point***

```
backpack = ("map", "compass", "torch")
backpack[0] = "magic map"
print(backpack)
```

- Prints ("magic map", "compass", "torch")
- (selected) Gives a TypeError
- Prints original backpack
- Gives a SyntaxError

Assessment submitted.
X

13) What will be the output? ***1 point***

```
pouch = {"coin", "potion", "coin"}
print(pouch)
```

Select all that apply:

- [x] The output will contain only unique elements
- [x] The output may appear in any order
- [ ] It will raise a syntax error due to duplicate elements
- [ ] The output will be a list with 2 elements

14) What will be the final contents of the pouch? ***1 point***

```
pouch = {"potion"}
pouch.add("gem")
pouch.remove("potion")
print(pouch)
```

- ( ) {'potion', 'gem'}
- (•) {'gem'}
- ( ) ['gem']
- ( ) Error due to remove()

15) What is the type of x in the following code? ***1 point***

```
x = ("compass")
print(type(x))
```

- ( ) < class 'tuple' >
- ( ) < class 'list' >
- (•) < class 'str' >
- ( ) < class 'set' >

16) Which of the following statements is **true**? ***1 point***

- ( ) Tuples can’t store duplicate items
- ( ) Sets maintain the order of elements
- (•) Sets automatically remove duplicate items
- ( ) Tuples can be changed after creation

17) What will this code output? ***1 point***

Assessment submitted.
X

```
backpack = ("map", "map", "torch")
pouch = {"map", "map", "torch"}
print(len(backpack), len(pouch))
```

- ○ 3 3
- ○ 2 2
- ◉ 3 2
- ○ 2 3

18) Dilrez tries to update a tuple. Find and fix the error: ***1 point***

```
gear = ("rope", "hook", "lamp")
gear[1] = "magic hook"
print(gear)
```

What's the issue?

- ○ Tuple index out of range
- ○ Syntax error in tuple
- ◉ Tuples are immutable
- ○ Need to use append()

19) What will be the output of the following code? ***1 point***

```
items = ["map", "torch", "torch", "map"]
unique_items = set(items)
fixed_items = tuple(unique_items)

print(len(items), len(unique_items), fixed_items)
```

- ☑ 4 2 ('map', 'torch')
- ☑ 4 2 ('torch', 'map')
- ☐ 2 2 ('map', 'torch')
- ☐ 4 4 ('map', 'torch', 'torch', 'map')

20) Dilrez does the following: ***1 point***

```
backpack = ("map", "torch")
pouch = {"torch", "map"}
supply_list = ["map", "torch"]
```

Which of the following is true based on this setup?

- ☐ All three containers contain the same items and in the same order

Assessment submitted.
X

- [x] Only the pouch automatically removes duplicates
- [x] backpack[1] and supply_list[1] give the same result
- [ ] Sets can be indexed just like lists and tuples

Himmat was given a task: Find the **factorial** of a number.

He read that:

The factorial of a number n is written as n!
It means:

n! = n × (n-1) × (n-2) × ... × 1

So, for example:

4! = 4 × 3 × 2 × 1 = 24

But instead of doing it manually, Himmat decided to write a **recursive function** — a function that calls **itself**.

```
def factorial(n):
  if n == 0:
    return 1
  return n * factorial(n - 1)
```

21) Why does the code return 1 when n == 0? ***1 point***

- ( ) Because 0 × anything is 0
- (•) To stop the recursion at some point
- ( ) By mistake; it should return 0
- ( ) It helps to calculate square of the number

22) What will be the return value of factorial(1)? ***1 point***

- ( ) 0
- (•) 1
- ( ) It will call itself infinitely
- ( ) 2

23) Identify the error in the following code: ***1 point***

```
def factorial(n):
    return n * factorial(n - 1)

print(factorial(3))
```

- ( ) It will return 6
- (•) No base case defined, will result in infinite recursion
- ( ) n is not decremented
- ( ) Syntax error

Assessment submitted.
X

24) What does the line n! = n × (n-1) × (n-2) × ... × 1 represent in the comprehension? ***1 point***

- ○ An infinite series
- ○ A summation of natural numbers
- ◉ A recursive definition of factorial
- ○ A non-recursive mathematical shortcut

25) ***1 point***

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return factorial(n - 1) * n
```

How many recursive calls are made when he executes factorial(4)?

- ○ 3
- ○ 4
- ◉ 5
- ○ Infinite

26) Himmat accidentally writes: ***1 point***

```
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n - 1)
```

What will happen if he calls factorial(0)?

- ○ Returns 1
- ○ Returns 0
- ◉ Throws a recursion error
- ○ Returns -1

27) What would be a correct way to **make Himmat's function handle negative numbers safely?** ***1 point***

- ○ Add return -1 for all inputs
- ◉ Add a base case if n < 0: return "Invalid input"
- ○ Remove recursion altogether
- ○ Multiply by -1 inside the function

28) If Himmat's function is modified as: ***1 point***

```
def factorial(n):
    if n == 0:
        return 1
    return factorial(n) * n
```

Assessment submitted.
X

What happens when factorial(3) is called?

- ○ Returns 6
- ○ Returns 0
- ◉ Infinite recursion
- ○ Error due to syntax

29) Himmat runs this code to reverse a string: ***1 point***

```
def reverse_string(s):
    if len(s) <= 1:
        return s
    return reverse_string(s[1:]) + s[0]

print(reverse_string("a"))
```

What will be the output?

- ◉ "a"
- ○ ""
- ○ Error (index out of range)
- ○ "aa"

30) Himmat wants to reverse the word "race" using recursion. He writes a function that checks ***1 point*** whether the string is non-empty as the base case.

```
def reverse_string(s):
    if s:
        return reverse_string(s[1:]) + s[0]
    else:
        return ""
reverse_string("race")
```

What will be the output of reverse_string("race")?

- ○ "race"
- ◉ "ecar"
- ○ "rcae"
- ○ Error due to slicing empty string

You may submit any number of times before the due date. The final submission will be considered for grading.

Submit Answers